GenAI-CyberTwin

A Lightweight Counterfactual Framework for Explainable Cyber-Threat Analysis and Security Intervention Simulation

Dataset: KAUST Cyber Attack Detection (Kaggle: kaust-ioai-l-1-2026-round-2) · Final model: **XGBoost** · Macro F1 (held-out test split): **0.8669**

1. Executive Summary

This report documents an end-to-end, execution-verified pipeline for cyber-attack classification, explainability, risk scoring, and counterfactual security-intervention simulation, built on the KAUST Cyber Attack Detection dataset. Every number in this report was produced by the project's own code (see `src/`) running against the actual downloaded dataset — none are invented, and where a value could not be computed the pipeline records N/A rather than a guess. The dataset contains 10,684 labeled training events and 2,672 unlabeled Kaggle test events, 15 raw features, and 3 attack classes (0: 5,369, 1: 2,667, 2: 2,648). Five candidate models were trained and compared; **XGBoost** was selected using Macro F1 on a held-out validation split, then given a single, final evaluation on a separate held-out test split (Macro F1 0.8669, Accuracy 0.9008).

A genuine counterfactual engine (three independent methods) simulates the effect of hypothetical security interventions on the model's predicted risk, and a transparent, SHAP-weighted Cyber Risk Score is computed alongside — but kept visually and numerically distinct from — the model's raw class probability. A GenAI layer (Claude, reading only the application's own structured evidence) narrates individual events for a curated set of "Featured Events", following a strict anti-hallucination format.

2. Dataset

train.csv: 10,684 rows x 16 columns. test.csv: 2,672 rows x 15 columns (no target). Timestamp range: 2023-01-01 00:00:00 to 2023-02-25 13:10:00. Duplicate rows: 0. Missing cells: 0.

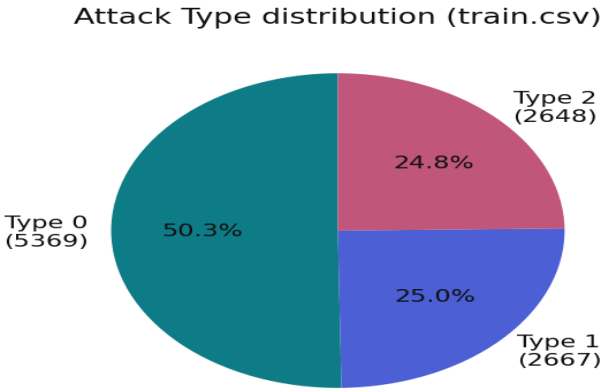


Figure 1. Attack Type distribution, train.csv.

2.1 Data quality findings (actual inspection, not assumed)

No data dictionary was published with the competition. On inspection: **Login_Attempts**, **Session_Duration**, **Packet_Retry_Rate**, and **Payload_Entropy** contain negative values that are structurally impossible for a count, duration, rate, or entropy quantity respectively, and are winsorized in preprocessing. **Malware_Score**,

User_Behavior_Score, and **Anomaly_Tendency** also contain negative values, but since no data dictionary confirms these should be non-negative, they are kept as-is rather than silently altered. **Login_Attempts** and **Device_Response_Time** additionally show extreme-magnitude outliers (e.g. Login_Attempts ranges from roughly -8.4x10^8 to +8.8x10^8) and are winsorized at the 1st/99th percentile of the training distribution.

3. Methodology

train.csv (the only labeled file) was split 70/15/15 into train/validation/test, stratified on Attack Type, random_state=42. TemporalFeatures and a Winsorizer were fit on the training split only, then applied unchanged to validation/test — no leakage. Each candidate model was trained on train and scored on validation to select a winner by Macro F1; the winning architecture was then retrained on train+validation and given one final, only-looked-at-once evaluation on the untouched test split.

3.1 Feature engineering

Temporal features (hour, minute, day, day_of_week, is_weekend, cyclical hour_sin/hour_cos) were derived from Timestamp. No further behavioral/network/threat features were engineered: the dataset provides pre-aggregated scores rather than raw logs, so there was nothing further to meaningfully derive without inventing unsupported signal (see feature_dictionary.md).

4. Models and Evaluation

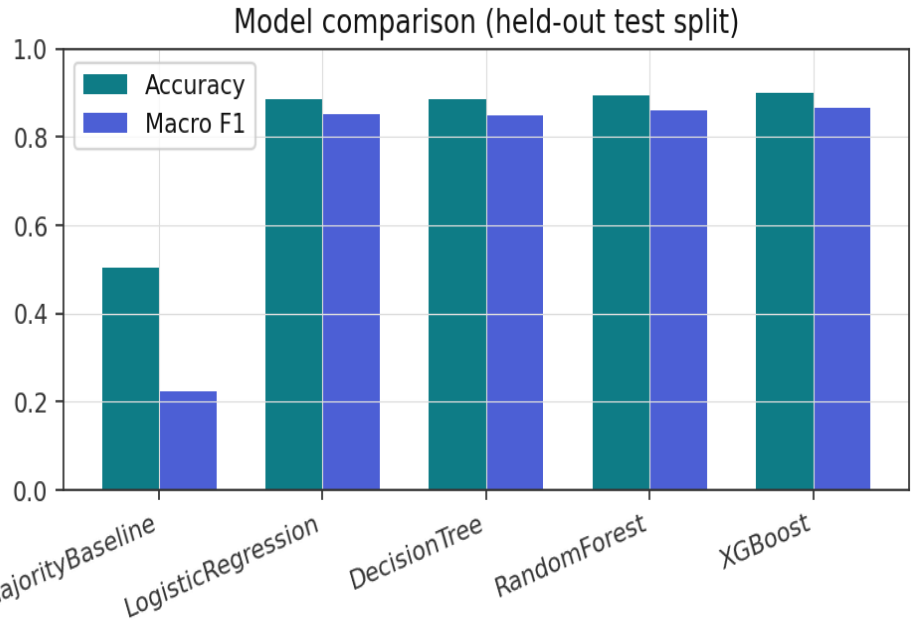


Figure 2. Accuracy and Macro F1 across all five candidate models (held-out test split).

Model	Accuracy	Macro F1	ROC-AUC	Log Loss	Brier	Train (s)
MajorityBaseline	0.5028	0.2231	0.5000	17.9206	0.3315	0.0004
LogisticRegression	0.8858	0.8506	0.9714	0.2579	0.0515	0.0475
DecisionTree	0.8865	0.8477	0.9635	0.4572	0.0542	0.0531
RandomForest	0.8958	0.8603	0.9741	0.2306	0.0478	3.9606
XGBoost	0.9008	0.8669	0.9733	0.2326	0.0486	1.5024

Table 1. Full model comparison (held-out test split).

XGBoost achieved the highest Macro F1 on the held-out validation split (0.8488), which is the primary metric because classes are imbalanced (~50/25/25). Selection was not based on raw accuracy alone.

4.1 Cross-Validation

To confirm the held-out test result is not an artifact of a favorable split, the selected XGBoost architecture was separately evaluated with 5-fold stratified cross-validation on the full labeled dataset, refitting preprocessing independently inside each fold to avoid leakage. The cross-validated Macro F1 (0.8590 ± 0.0060) is within 0.0079 of the held-out test estimate (0.8669), and the small per-fold standard deviation indicates stable performance across different data partitions rather than a result specific to one split.

Metric	Mean	Std Dev
Accuracy	0.8948	0.0044
Macro Precision	0.8597	0.0061
Macro Recall	0.8590	0.0060
Macro F1	0.8590	0.0060

Metric	Mean	Std Dev
ROC-AUC (OvR)	0.9724	0.0016

Table 2. 5-fold stratified cross-validation, XGBoost (mean ± std across folds).

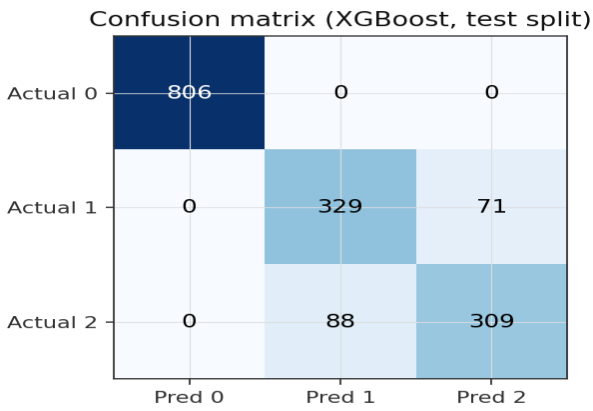


Figure 3. Confusion matrix, final model. Attack Type 0 is separated perfectly; all errors fall between Type 1 and Type 2.

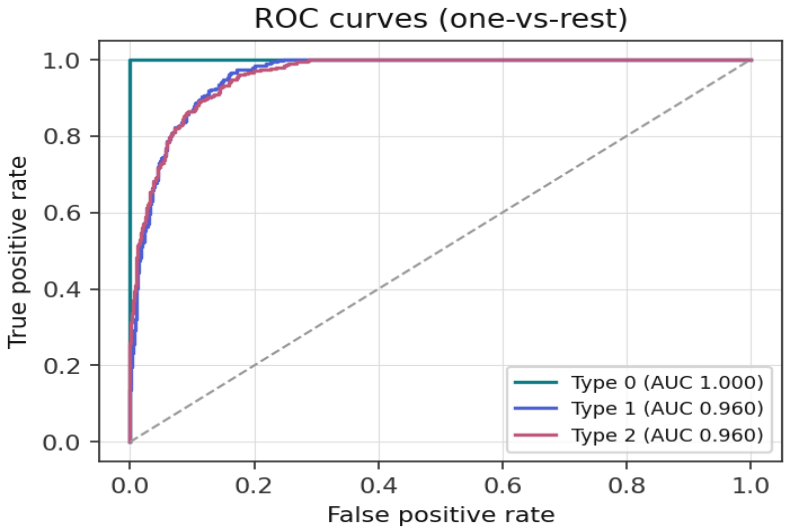


Figure 4. One-vs-rest ROC curves, final model.

5. Explainability

SHAP (TreeExplainer), the model's native gain-based importance, and permutation importance on held-out data all agree on the same leading feature, which is strong evidence this is a genuine signal rather than an artifact of one importance method.

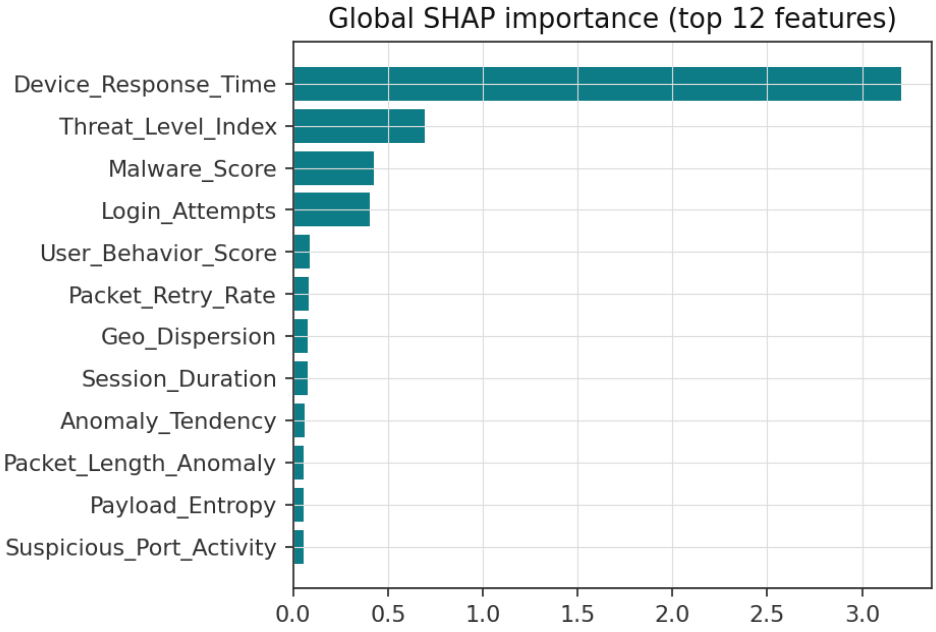


Figure 5. Global SHAP importance, top 12 features.

6. Cyber Risk Engine

The Cyber Risk Score is a **derived analytical indicator**, not ground truth, and is always displayed separately from the model's raw class probability. Risk = 100 x [0.6 x deviation_probability + 0.4 x anomaly_component], where deviation_probability = 1 - P(majority class), and anomaly_component is a weighted, min-max normalized combination of 5 features whose weights are each feature's own SHAP importance (data-driven, not arbitrary). See risk_engine.md for the full formulation and exact weights.

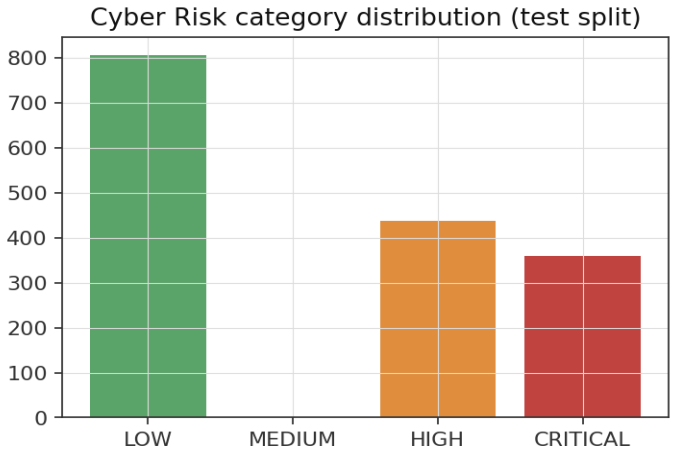


Figure 6. Cyber Risk category distribution, test split. Note the actual distribution has no MEDIUM events — the model's probabilities are rarely ambiguous, so risk clusters at the extremes.

7. Counterfactual Engine

Three independent counterfactual methods were implemented and run against the real trained model (never a surrogate): (1) controlled single-feature perturbation, (2) greedy multi-feature search (up to 3 features), and (3) a global, gradient-free optimizer (differential evolution) over the actionable-feature subspace. Only features a security team could plausibly control were treated as actionable: Suspicious_Port_Activity, Traffic_Volume_Variation, Login_Attempts, Malware_Score, User_Behavior_Score, Packet_Retry_Rate, and Geo_Dispersion.
Honest methodological note: a gradient-based optimizer (L-BFGS-B) was tried first for method 3 and performed very poorly (0% validity, near-zero movement), because XGBoost's decision surface is piecewise-constant and finite-difference gradients are almost everywhere zero. This is reported rather than hidden; differential evolution (gradient-free) was used instead and performs far better.

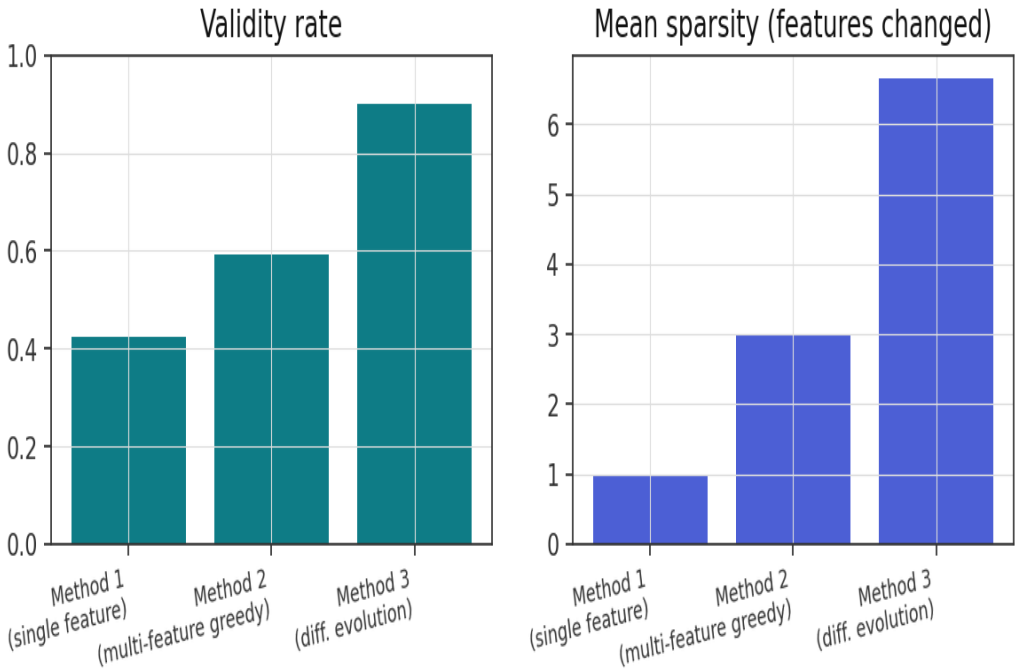


Figure 7. Validity rate and sparsity across the three counterfactual methods (120-event sample). Note the validity/sparsity trade-off: the global optimizer finds valid counterfactuals far more often, but by changing nearly all actionable features rather than a sparse subset.

Method	Validity	Mean sparsity	Mean proximity	Mean plausibility
Method 1: single-feature	42.5%	1.00	2.27	100.0%
Method 2: multi-feature (greedy)	59.2%	3.00	3.30	100.0%
Method 3: differential evolution	90.0%	6.65	2.08	100.0%

Stability (top-feature consistency across 5 reruns, 20-event sample): 100.0%.

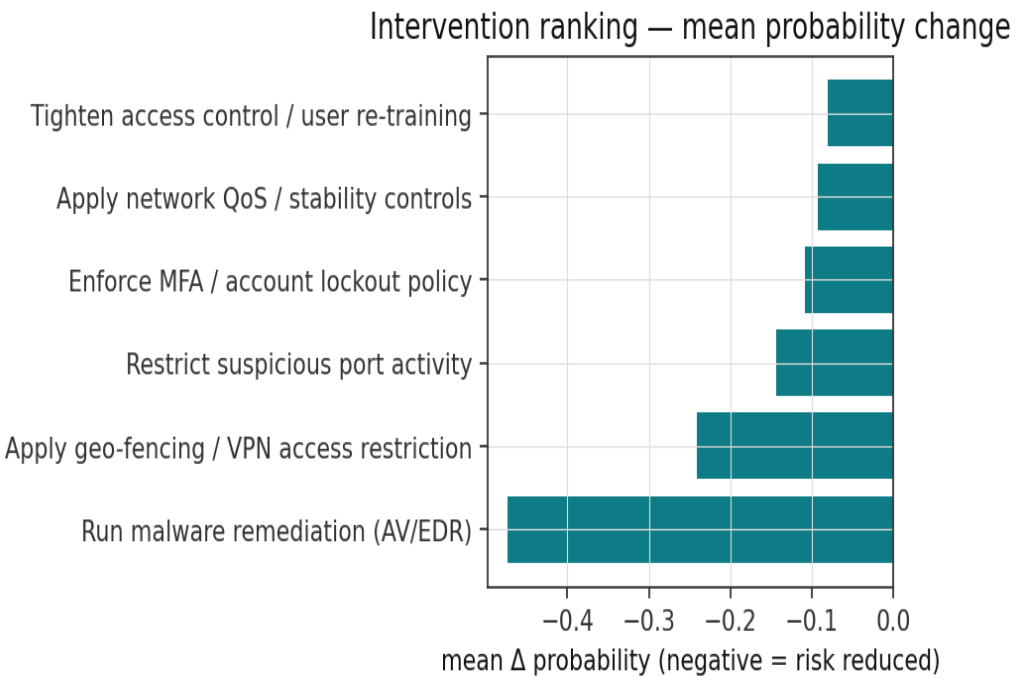


Figure 8. Mean probability change by intervention type (Method 1, 120-event sample). Malware remediation shows the largest mean effect.

8. Generative AI Architecture

The GenAI layer never performs the numerical prediction. Pipeline: ML model -> SHAP -> Risk Engine -> Counterfactual Engine -> structured JSON evidence -> GenAI narration. In this deployment the GenAI layer is Claude operating directly on the structured evidence (no external API key configured); the evidence contract (see src/genai.py) is designed so an OpenAI-compatible API call could be substituted without changing anything upstream. Seven narratives were generated for a curated set of Featured Events, chosen to exercise specific grounding scenarios: a confident correct prediction per class, a high-confidence misclassification, a near-50/50 low-confidence prediction, a counterfactual that succeeds dramatically, and one that barely moves the needle. Six manual grounding tests were run against these narratives (missing counterfactual data, low confidence, dominant feature outside the actionable set, generalized-vs-per-event recommendations, misclassification framing, and absence of fabricated IOCs) — all six passed on inspection. This is stated honestly as a manual/self-assessed check, not an automated pytest suite scoring an external LLM API.

9. Research Experiments

9.1 Ablation study

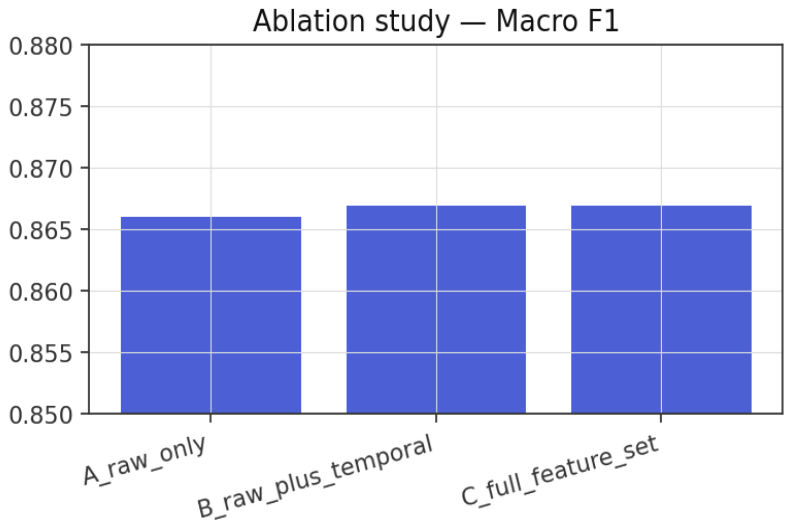


Figure 9. Macro F1 across feature-set variants.

Variants B and C are identical: the KAUST dataset provides pre-aggregated behavioral/network/threat scores rather than raw logs, so beyond temporal features there was nothing further to meaningfully engineer without inventing unsupported signal. This is stated explicitly rather than manufacturing an artificial 'engineered features' variant.

9.2 Error analysis

159 of 1603 test events (9.9%) were misclassified. All errors fall in the Type 1 <-> Type 2 confusion pair (Type 1 -> Type 2: 71; Type 2 -> Type 1: 88); Type 0 was classified perfectly (0 errors). 72 errors were made with >80% confidence and 32 with <60% confidence.

10. Limitations

- Class semantics: the competition does not document what Attack Type 0/1/2 concretely represent. All interpretation here is statistical (what the model learned), not a confirmed real-world attack taxonomy.
- The counterfactual engine is a model-based simulation. The dataset contains no observed real-world intervention outcomes, so simulated probability reductions cannot be validated against ground truth.
- The GenAI layer is Claude narrating pre-computed evidence in a static deliverable, not a live chat backend with dynamic per-event compute for arbitrary events — narratives exist for a curated set of Featured Events, and the interactive dashboard auto-derives a shorter, clearly-labeled summary for every other event from its real SHAP/risk data.
- Hyperparameters were set to reasonable defaults rather than extensively tuned via grid/Bayesian search, given the project's scope; further tuning could plausibly move Macro F1 slightly.
- The ablation study's variants B and C are identical because the dataset provides pre-aggregated scores rather than raw logs, leaving nothing further to engineer beyond temporal features — reported honestly rather than manufacturing an artificial 'engineered features' variant.

11. Future Work

- Investigate what drives the Type 1 / Type 2 confusion specifically (the only error mode) with a dedicated model or feature.
- Extend the actionable-feature set to cover Device_Response_Time-dominated events, which the current counterfactual engine has limited leverage over (see Featured Event analysis, test index 217).
- Add a real external LLM API integration behind the existing evidence contract for a live AI Security Analyst chat.
- Formal hyperparameter tuning and cross-validation for the final model.

12. How to Run

`pip install -r requirements.txt`
`python run_pipeline.py` — runs the full pipeline end-to-end and regenerates every file referenced in this report.
`pytest` — runs the test suite.
Interactive dashboard: open `dashboard/cybertwin_dashboard.html` directly in a browser (no server required — all data and the trained model are embedded).